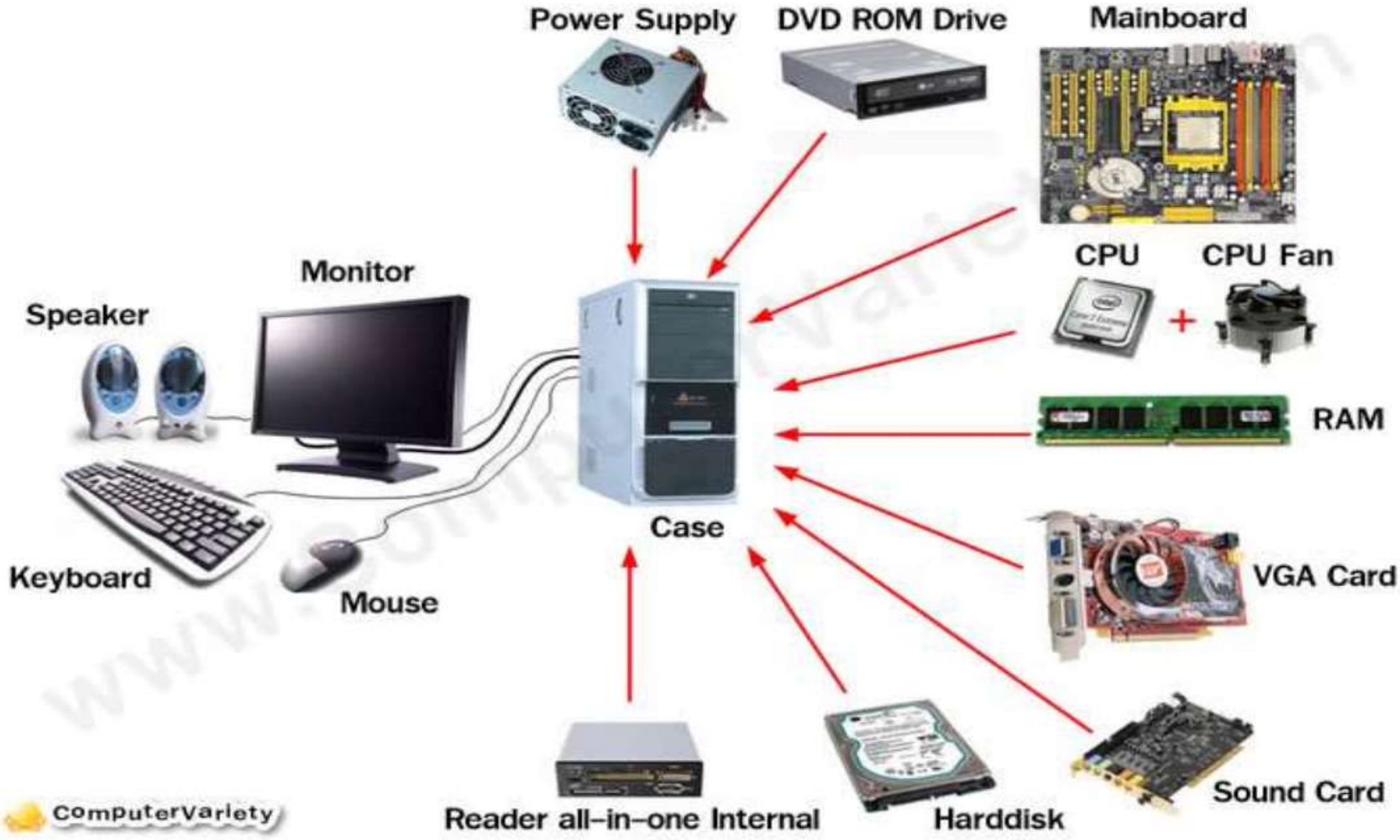
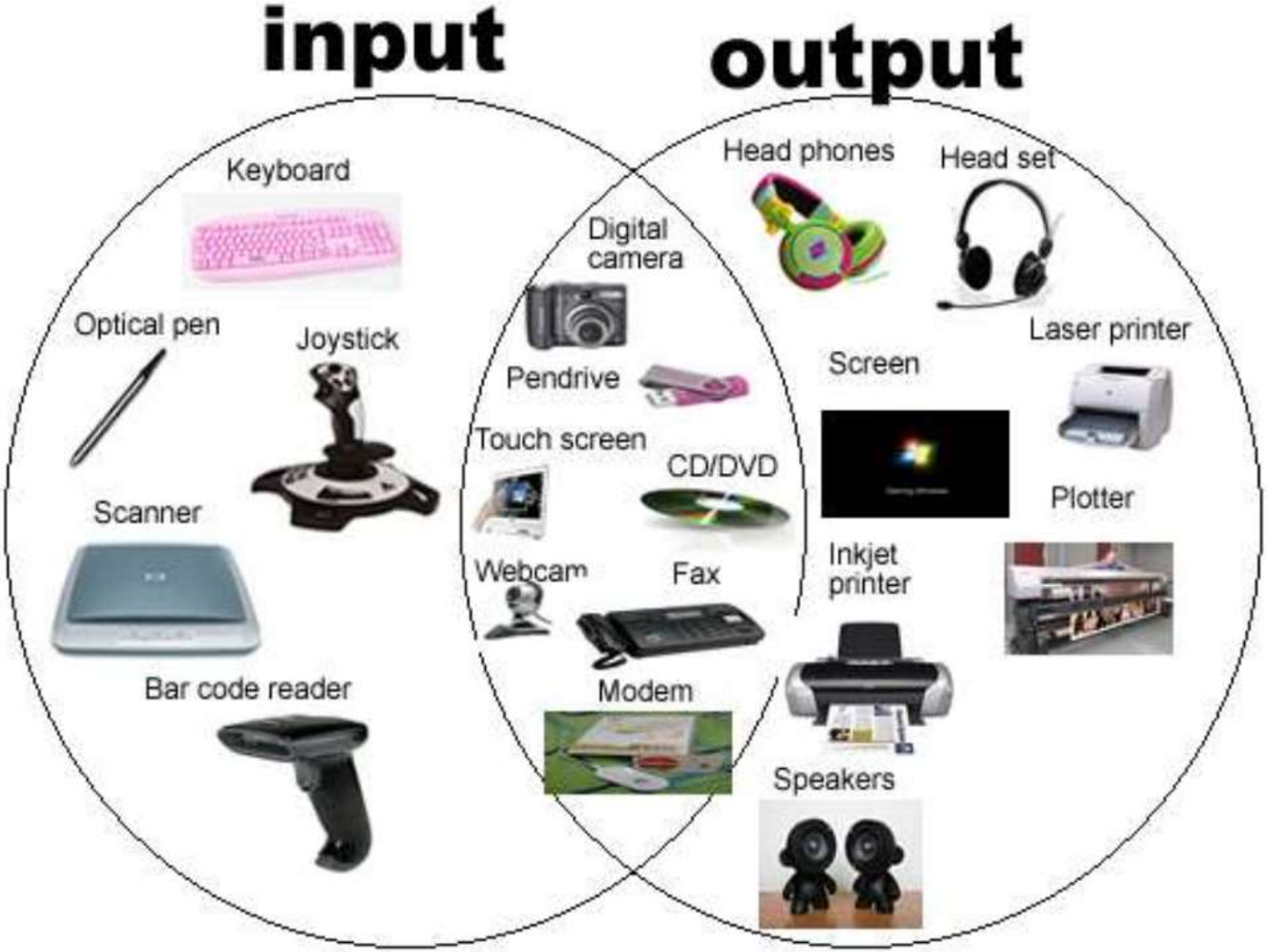
Structure & Function

- Structure is the way in which components relate to each other
- Function is the operation of individual components as part of the structure
- All computer functions are:
  - Data processing
  - Data storage
  - Data movement
  - Control

Computer System: Hardware



Computer System: Peripherals



Computer System: Software

Systems software

Applications software

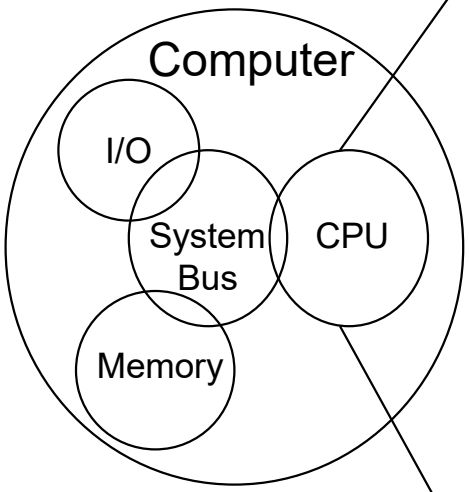


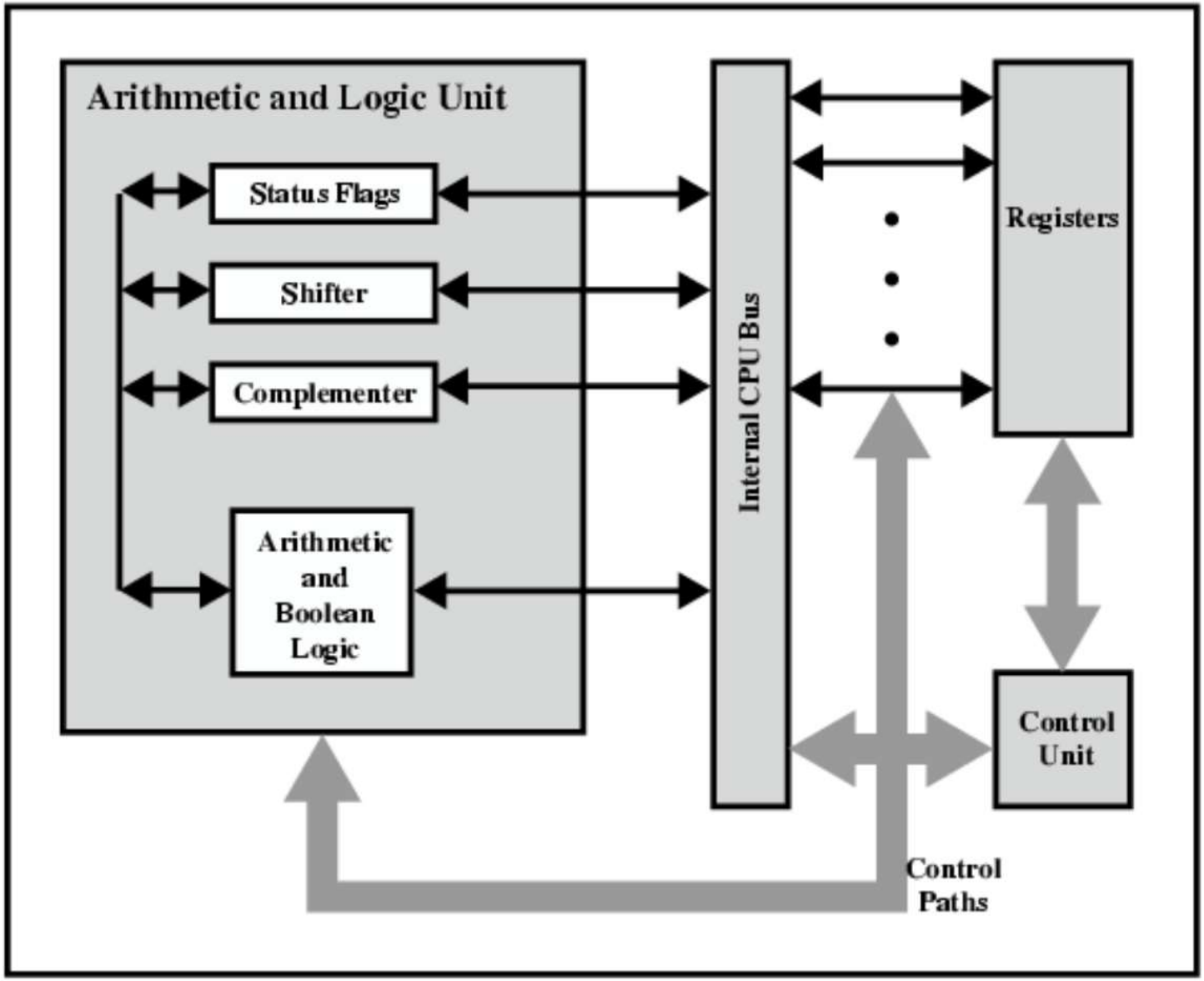


Foundation of CA

- Instruction set architecture
- Taxonomy / classification of computers and ISA
- Addressing modes
- RISC and CISC architectures
- Performance of different computer architectures

Structure - The CPU





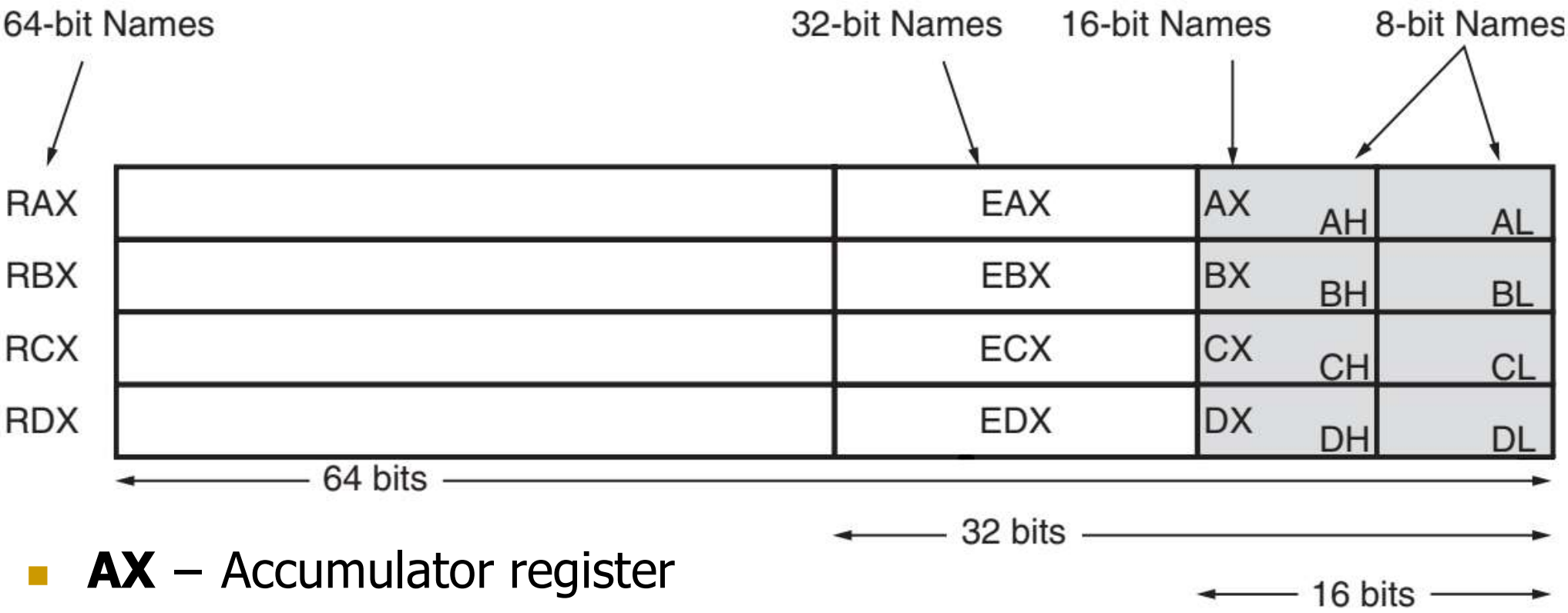
Processor Registers

- General registers
  - Store any transient data required by the program
- Control registers
  - Changes or controls the general behavior of a CPU
- Segment registers
  - Stores the starting address of the different segments defined in the program execution

General purpose registers:

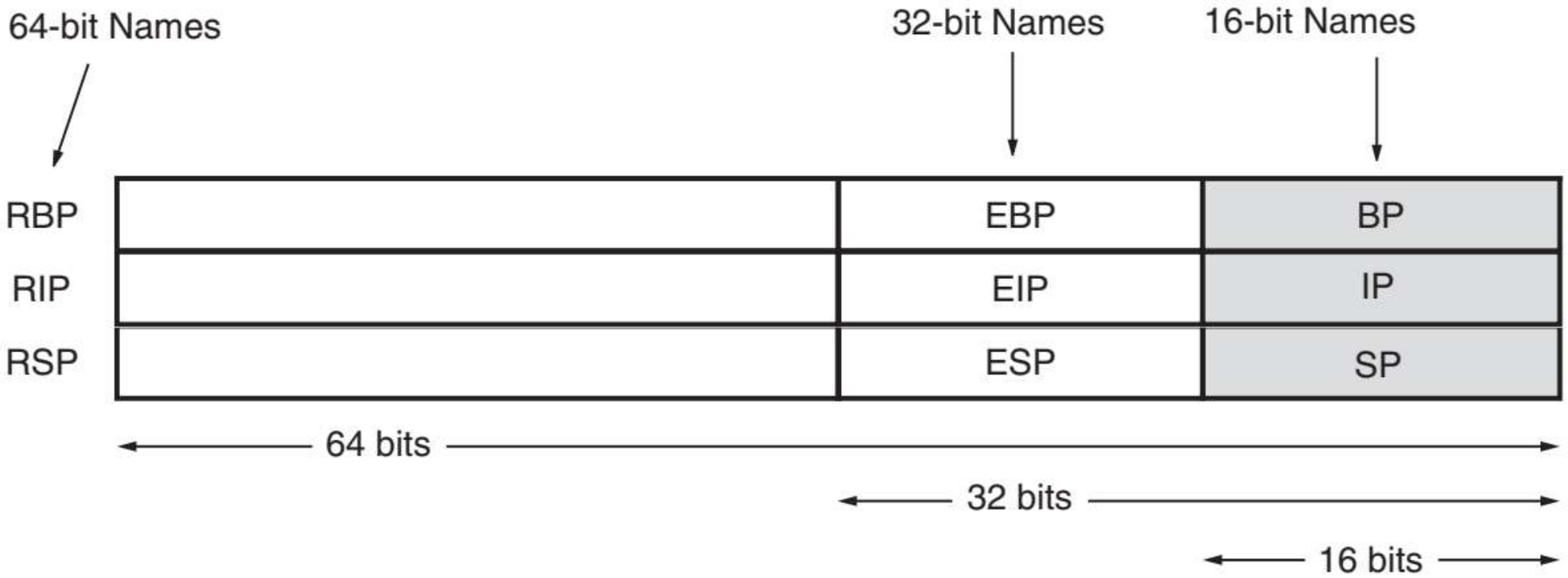
- Data Registers
- Pointer registers
- Index registers

General registers: **Data Registers**



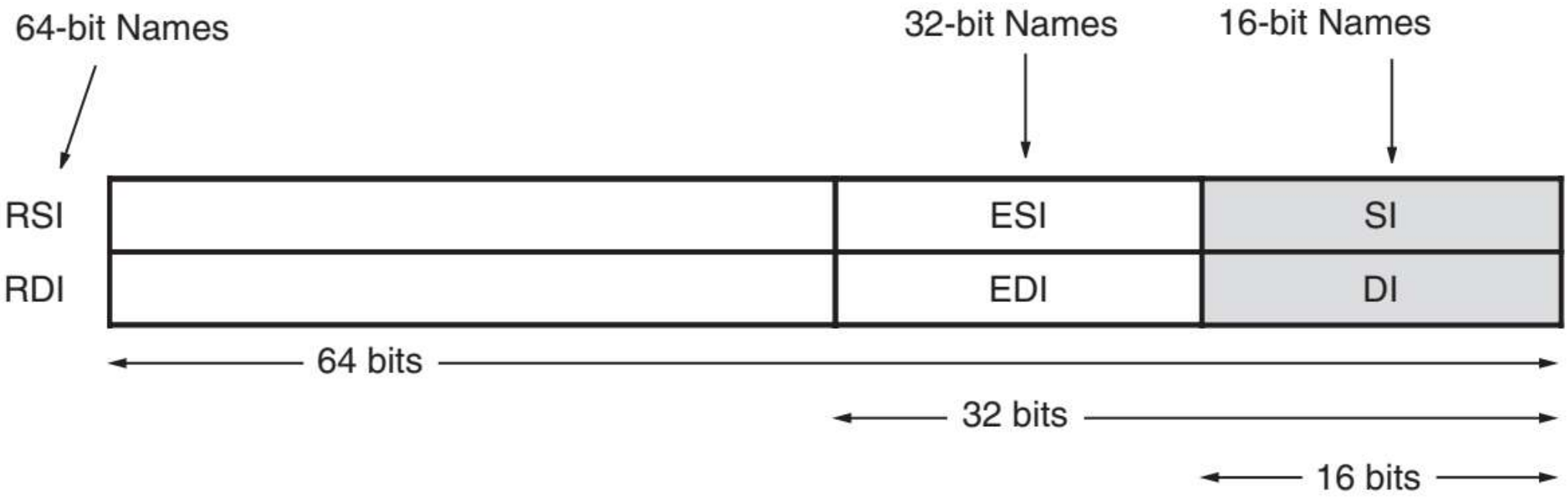
- **AX** – Accumulator register
- **BX** – Base register
- **CX** – Counter register
- **DX** – Data register
- **E** := Extended, **R** := Register (mnemonic for 64-bit registers)
- **L** := register lower 8-bits, **H** := register higher 8-bits in 16-bit part

General registers: **Pointer registers**



- **BP** – Base Pointer (16-bit register): referencing the parameter variables passed to a subroutine
- **SP** – Stack Pointer (16-bit register): provides the offset value within the program stack
- **IP** – Instruction Pointer (16-bit register): offset address of the next instruction to be executed

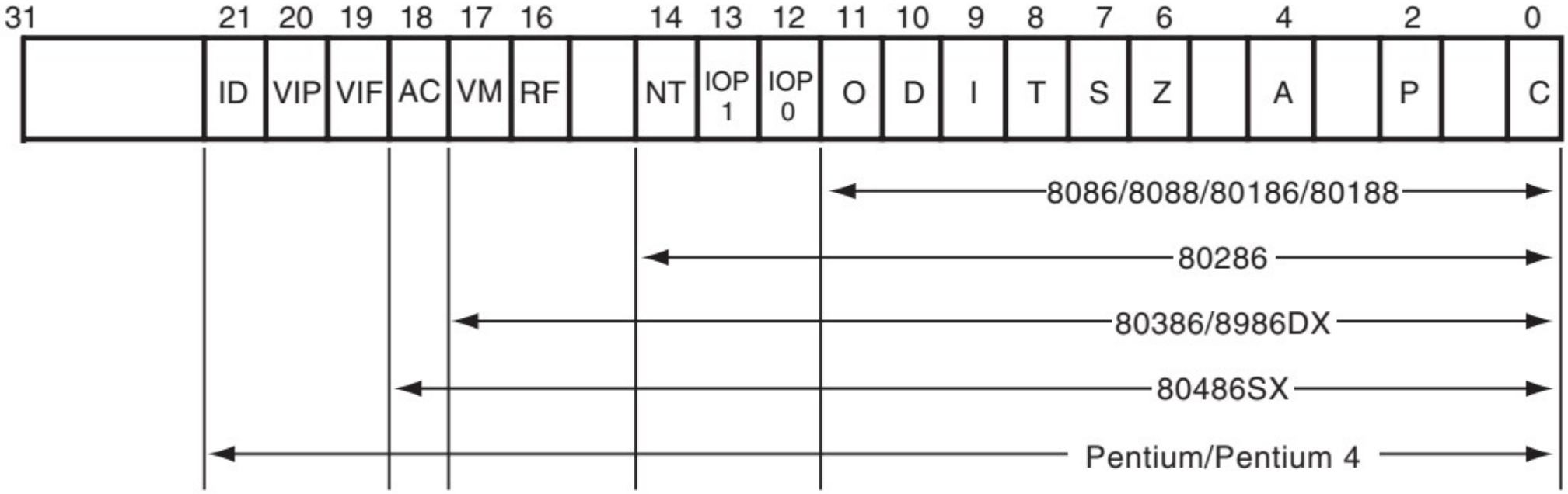
General registers: **Index Registers**



- **SI** – Source Index (16-bit register).
- **DI** – Destination Index (16-bit register).

NEW LECTURES

Control Register: Flag register



- **Carry Flag (C)** – It contains the carry of 0 or 1 from a high-order bit (leftmost) after an arithmetic operation.
- **Parity Flag (P)** – It indicates the total number of 1-bits in the result obtained from an arithmetic operation. An even number of 1-bits clears the parity flag to 0 and an odd number of 1-bits sets the parity flag to 1.

Flag register continued…

- **Auxiliary Carry Flag (AC)** − It contains the carry from bit 3 to bit 4 following an arithmetic operation; used for specialized arithmetic. The AF is set when a 1-byte arithmetic operation causes a carry from bit 3 into bit 4.
- **Zero Flag (Z)** − It is set if the last logical or mathematical instruction has produced a zero in its destination.
- **Sign Flag (S)** − It shows the sign of the result of an arithmetic operation. The sign is indicated by the high-order of leftmost bit. This flag is set to 1 if result is negative otherwise it is clear to 0.
- **Trap Flag (T)** − It allows setting the operation of the processor in single-step mode. The DEBUG program we used sets the trap flag to 1, so we could step through the execution one instruction at a time.

Flag register continued…

- **Interrupt Flag (I)** − It disables the external interrupt to processor when the value is 0 and enables CPU to be interrupted when set to 1.
- **Direction Flag (D)** − determines either the increment or decrement mode for the DI and/or SI registers during string instructions. If D=1, the registers are automatically decremented (bottom-up memory access); if D=0, the registers are automatically incremented (top-down memory access).

  Overflow Flag (O) − It indicates the overflow of a high-order bit (leftmost bit) of data after a signed arithmetic operation.
- **I/O Privilege Level field (IOPL):** it defines privilege level of current process.

Flag register continued…

- **Nested Task (NT):** it controls execution of chained interrupts. It is set if current process is linked to the next process.
- **RF (resume)**: it is used with debugging to control the resumption of execution after the next instruction.
- **VM (virtual mode)**:VM flag bit selects virtual mode operation in a protected mode system.
- **AC (Alignment Check)**: flag bit activates if a word or doubleword is addressed on a non-word or non-doubleword boundary.
- **VI (Virtual Interrupt) Flag**: it is a copy of the interrupt flag bit available to the Pentium-Pentium 4 microprocessors

Flag register continued…

- **VIP (Virtual Interrupt Pending) Flag**: it is used in multitasking environments in Pentium–Pentium 4 microprocessors to provide the operating system with virtual interrupt flags and interrupt pending information.
- **ID (identification) Flag**: it indicates that the Pentium–Pentium 4 microprocessors support the CPUID instruction. The CPUID instruction provides the system with its microprocessor information like its version number and manufacturer.
-

CPU Registers

- Program Counter (PC)
  - An incrementing counter that holds the memory address of the instruction that is to be executed next
- Memory Address Register (MAR)
  - Holds the address of a block of memory for reading from or writing to
- Memory Buffer Register (MBR)
  - A two-way register that holds data fetched from memory (and ready for the CPU to process) or data waiting to be stored in memory. It is also called as memory data register.

CPU Registers

- Instruction Register (IR)
  - holds the instruction currently being executed or decoded.
- Instruction Buffer Register (IBR)
  - temporary register where the opcode of the currently fetched instruction is stored
- Accumulator (AC)
  - Performs Arithmetic and logical operations. It holds temporarily operands and results of ALU operations.
- Multiplier Quotient (MQ)
  - It holds temporarily operands and results of ALU operations.
  - For example, the result of multiplying two 40-bit numbers is an 80-bit number; the most significant 40 bits are stored in the AC and the least significant in the MQ.

Segment Registers

- Segment registers point to starting location in memory segment.
- **CS (Code Segment)** – A 16-bit CS register stores the starting address of the code segment part in memory.
- **DS (Data Segment)** – A 16-bit DS register stores the starting address of the data segment that contains data, constants and work areas.
- **SS (Stack Segment)** – SS register stores the starting address of the stack that contains data and return addresses of procedures or subroutines.
- **ES(Extra Segment)** – ES register points to extra data, i.e., an additional data segment that is used by some of the string instructions to hold destination data.

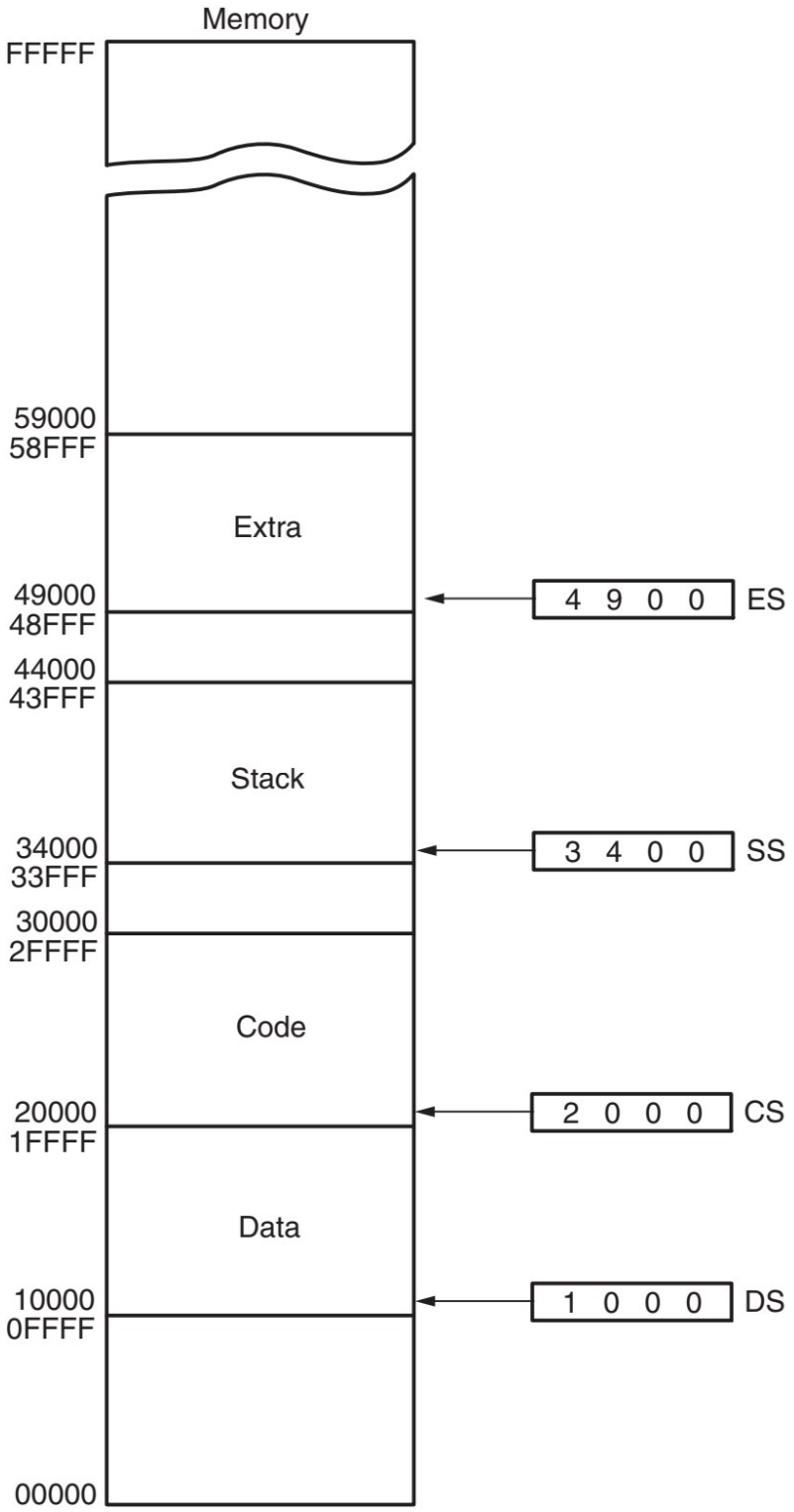
Physical address generation

Default 32-bit segment and offset combinations

Segment	*Offset*	*Special Purpose*
CS	EIP	Instruction address
SS	ESP or EBP	Stack address
DS	EAX, EBX, ECX, EDX, ESI, EDI, an 8- or 32-bit number	Data address
ES	EDI for string instructions	String destination address
FS	No default	General address
GS	No default	General address

Physical Address Calculation

- Offset is by default connected to a segment register as:



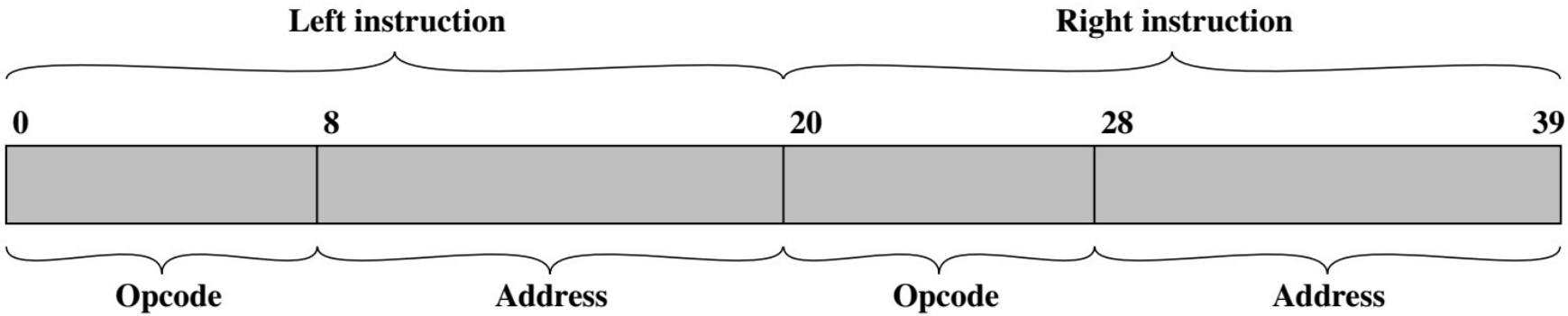
Register size effect

- 8-bit register can hold 1 byte data and processor can access memory of size 256,
- 16-bit registers can hold a word (2 byte data) and processor can access 64k memory,
- 32-bit registers can hold double word (4-byte data) and 4GB memory can be accessed,
- 64-bit registers can hold quad word (8-byte data) and 18-Billion GB memory can be accessed.

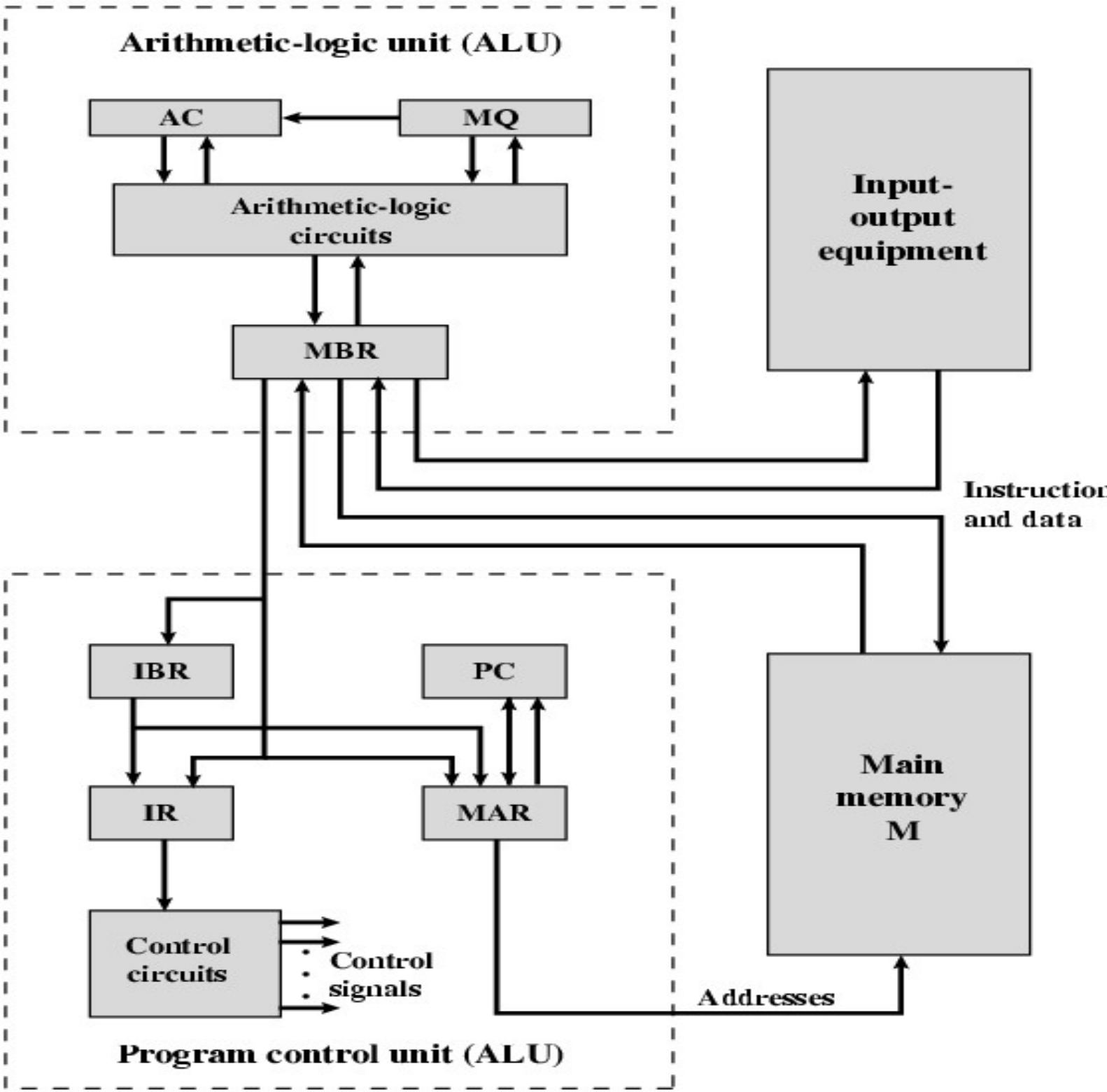
INSTRUCTION SET ARCHITECTURE: ABSTRACT OVERVIEW

ISA ?

- *Instruction Set Architecture* (ISA) refers to the actual programmer visible instruction set that serves as the boundary between the software and hardware.
- ISA consists of 1000 storage locations, called **words**, of 40 binary digits (bits) each.



Structure of ISA – detail



Instruction Groups

- Data Movement Instruction
  - Move, Push, POP
- Arithmetic and Logical Instructions
  - Add, Subtract, AND, OR
- Program Control Instruction: controls program execution, i.e., a set of instructions which control our instruction pointer (PC) register
  - If sequential execution of the program instructions or perform jump instructions etc.
    - cmp ax, 0
    - jne 1234
- Special Instructions: a set of instructions to change the processor state or behavior of CPU.
  - cli sti ; clear / set interrupt flag

Instruction Set (IS)

- An instruction defines 4 features as:
  1. What operation to perform like ADD or MOVE is first part of an instruction. This part defines either data transfer, or arithmetic or a logic operation,
  2. Second part tells information about operand, i.e., instruction will tell from where to fetch the operand like from register or from a memory location,
  3. Third part specifies the destination address of the CPU operation. For example, after adding two numbers where should the result store.
  4. Last part defines what next instruction needs to be executed, by default this is address of next instruction in the queue. However, in case of jump operation, this will specify the address of the next instruction specified by the jump operation.

Classes of IS based on addresses

- Note that 0~4 addresses need to be specified in the instructions
- Instructions are classed into 5 classes as:
  - 4-address instructions
  - 3-address instructions
  - 2-address instructions
  - 1-address instructions
  - 0-address instructions (Stacked based machine)

4-address instructions

2^8 = 256 instructions possible

2^{24} = 16 MB Memory accessible

1-Byte	3 Bytes	3 Bytes	3 Bytes	3 Bytes
Operation Code	Destination address	Operand 1 address	Operand 2 address	Next instruction address

- Instruction size = 1+3+3+3+3 = 13 bytes
- Note that if data bus is of 1-byte, maximum of 1 byte data is fetched from memory to CPU in one clock cycle.
- Number of bytes accessed from memory:
  - 13 for instruction fetch + 6 bytes for operands + 3 bytes for storing result in destination = 22 bytes

3-address instructions

1-Byte	3 Bytes	3 Bytes	3 Bytes
Operation Code	Destination address	Operand 1 address	Operand 2 address

- Instruction size = 1+3+3+3 = 10 bytes
- Number of bytes accessed from memory:
  - 10 for instruction fetch + 6 bytes for operands + 3 bytes for storing result in destination = 19 bytes

2-address instructions

1-Byte	3 Bytes	3 Bytes
Operation Code	Operand 1 + Destination address	Operand 2 address

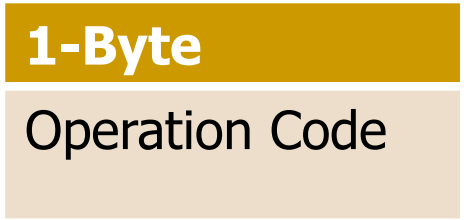
- Instruction size = 1+3+3 = 7 bytes
- Number of bytes accessed from memory:
  - 7 bytes for instruction fetch + 6 bytes for operands + 3 bytes for storing result in destination = 16 bytes

1-address instructions

1-Byte	3 Bytes
Operation Code	Operand 2 address

- Accumulator based machine has one operand stored in the accumulator register in CPU while second operand is fetched from location specified in source part.
- Instruction size = 1+3 = 4 bytes
- Total memory access bytes = 7 {4 + 3 (Operand 2 fetch) + 0 for storing result as the result is stored back in the accumulator register}

0-address instructions

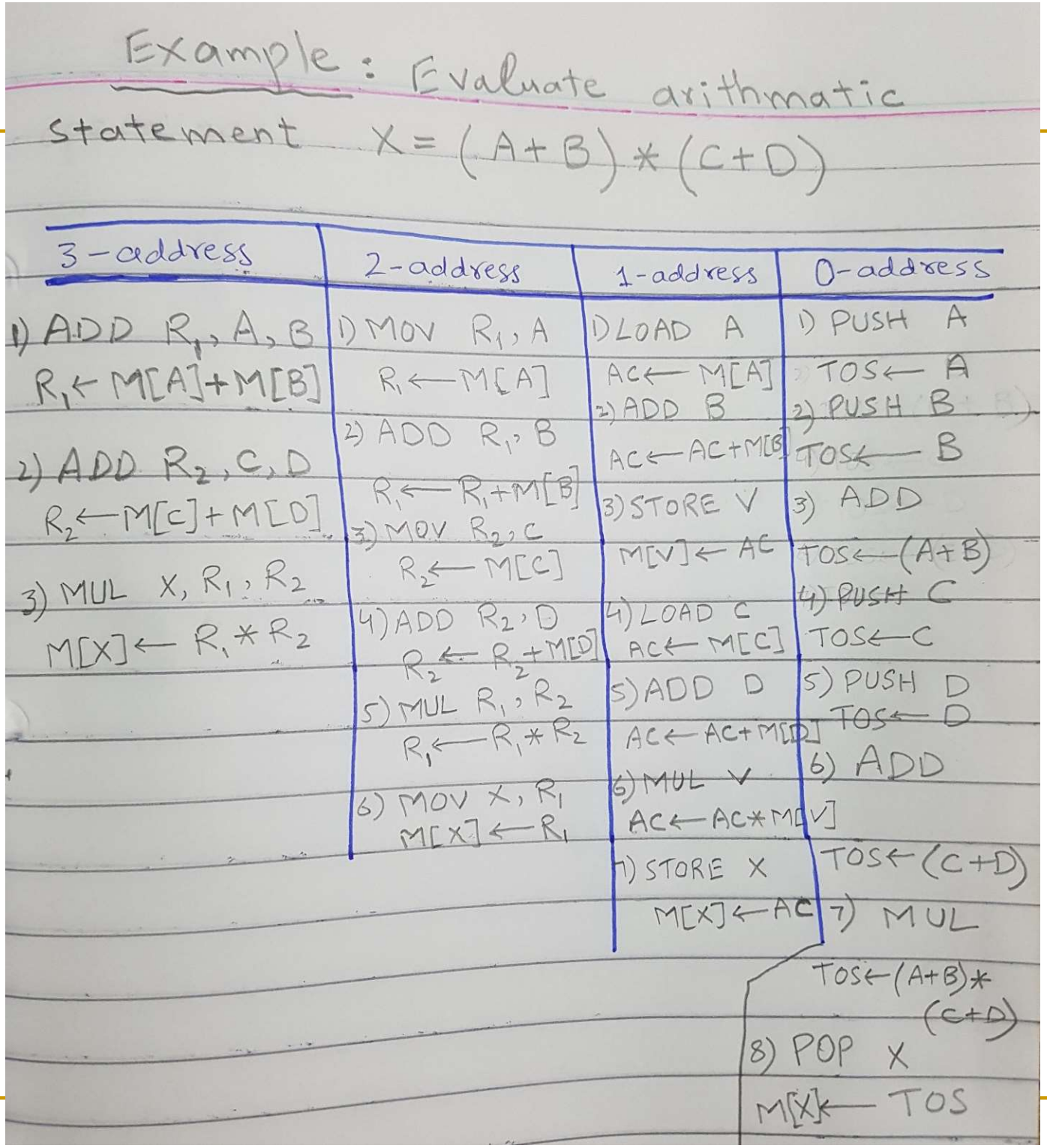


- Stacked based machine takes all the operands from the top of the stack and the result is also stored back on top of the stack, therefore, there is no need for source addresses.
- Total memory bytes access: 10 {1+ 6 (operands access from stack) + 3 (storing destination)}

Summary

Instruction Format	Code size	Number of memory bytes
4-address instruction	13	22
3-address instruction	10	19
2-address instruction	7	16
1-address instruction	4	7
0-address instruction	1	10

Example


Example: Evaluate arithmatic
Statement X = (A+B) * (C+D)
3-address
1) ADD R1, A, B
R1 ← M[A] + M[B]
2) ADD R2, C, D
R2 ← M[C] + M[D]
3) MUL X, R1, R2
M[X] ← R1 * R2
2-address
1) MOV R1, A
R1 ← M[A]
2) ADD R1, B
R1 ← R1 + M[B]
3) MOV R2, C
R2 ← M[C]
4) ADD R2, D
R2 ← R2 + M[D]
5) MUL R1, R2
R1 ← R1 * R2
6) MOV X, R1
M[X] ← R1
1-address
1) LOAD A
AC ← M[A]
2) ADD B
AC ← AC + M[B]
3) STORE V
M[V] ← AC
4) LOAD C
AC ← M[C]
5) ADD D
AC ← AC + M[D]
6) MUL V
AC ← AC * M[V]
7) STORE X
M[X] ← AC
0-address
1) PUSH A
TOS ← A
2) PUSH B
TOS ← B
3) ADD
TOS ← (A+B)
4) PUSH C
TOS ← C
5) PUSH D
TOS ← D
6) ADD
TOS ← (C+D)
7) MUL
TOS ← (A+B) * (C+D)
8) POP X
M[X] ← TOS

ADDRESSING MODES

Addressing Mode

- The ways addresses are specified by instructions are called *addressing modes.*
- It specifies a rule for interpreting or modifying the address field of the instruction before the operand is actually referenced.
- Arithmetic & Logic operations done by the instructions
- Data movement instructions in form of communication with memory or I/O
- Program flow instructions

Where Are the Operands?

Register addressing mode

* Most efficient way of specifying an operand
  - » operand is in an internal register

Examples

```
mov     EAX,EBX
mov     BX,CX
```

* The **`mov`** instruction

```
mov   destination,source
```

copies data from **`source`** to **`destination`**

Where Are the Operands? (cont'd)

Immediate addressing mode

*Data is part of the instruction

» operand is located in the code segment along with the instruction

» Efficient as no separate operand fetch is needed

» Typically used to specify a constant

Example

```
mov    AL,75
```

*This instruction uses register addressing mode for specifying the *destination* and immediate addressing mode to specify the *source*

Where Are the Operands? (cont'd)

Direct addressing mode

∗Data is in the data segment

» Need a logical address to access data

–Two components: segment:offset

» Various addressing modes to specify the offset component

–offset part is referred to as the ***effective address***

∗Offset is specified directly as part of the instruction

∗We write assembly language programs using memory labels (e.g., declared using DB, DW, LABEL,...)

» Assembler computes the offset value for the label

– Uses symbol table to compute the offset of a label

Where Are the Operands? (cont'd)

Direct addressing mode

Examples

`mov AL,result`

» Assembler replaces **`result`** by its effective address

`mov table1,56`

» **`table1`** is declared as

```
table1    DW   20 DUP (0)
```

» Since the assembler replaces **`table1`** by its effective address, this instruction refers to the first element of **`table1`**

– In C, it is equivalent to

```
table1[0] = 56
```

Where Are the Operands? (cont'd)

Direct addressing mode

•Problem with direct addressing

∗Useful only to specify simple variables

∗Causes serious problems in addressing data types such as arrays

» As an example, consider adding elements of an array

–Direct addressing does not facilitate using a loop structure to iterate through the array

–We have to write an instruction to add each element of the array

Where Are the Operands? (cont'd)

Indirect addressing mode

- The offset is specified indirectly via a register
  - Sometimes called *register indirect* addressing mode
  - For 16-bit addressing, the offset value can be in one of the three registers: BX, SI, or DI
  - For 32-bit addressing, all 32-bit registers can be used

Example

```
mov     AX,[BX]
```

- Square brackets [ ] are used to indicate that BX is holding an offset value
  - » BX contains a pointer to the operand, not the operand itself

Where Are the Operands? (cont'd)

- Using indirect addressing mode, we can process arrays using loops

Example: Summing array elements

* Load starting address (i.e., offset) of the array into BX
* Loop for each element in the array

» Get the value using the offset in BX

– Use indirect addressing

» Add the value to the running total

» Update the offset in BX to point to the next element of the array

Where Are the Operands? (cont'd)

Loading offset value into a register

- Suppose we want to load BX with the offset value of **`table1`**
- We cannot write

```
mov   BX,table1
```

- Two ways of loading offset value

» Using OFFSET assembler directive

– Executed only at the assembly time

» Using **`lea`** instruction

– This is a processor instruction

– Executed at run time

Where Are the Operands? (cont'd)

Loading offset value into a register

- Using OFFSET assembler directive
  - The previous example can be written as

```
mov   BX,OFFSET table1
```

- Using **`lea`** (load effective address) instruction
  - The format of **`lea`** instruction is

```
lea   register,source
```

  - The previous example can be written as

```
lea   BX,table1
```

Where Are the Operands? (cont'd)

Loading offset value into a register

Which one to use -- OFFSET or **`lea`**?

*Use OFFSET if possible

» OFFSET incurs only one-time overhead (at assembly time)

» **`lea`** incurs run time overhead (every time you run the program)

*May have to use **`lea`** in some instances

» When the needed data is available at run time only

– An index passed as a parameter to a procedure

» We can write

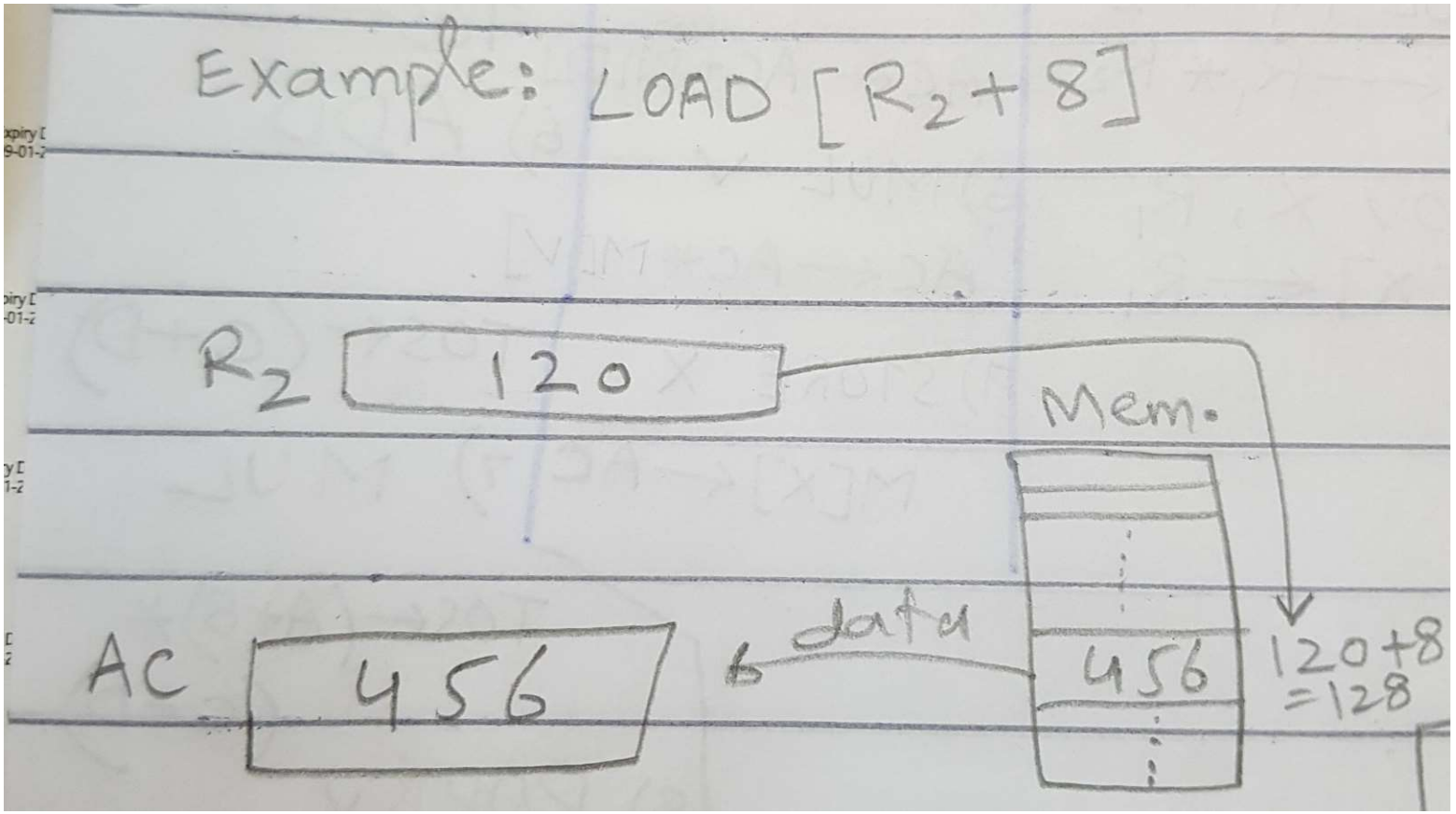
```
lea BX,table1[SI]
```

to load BX with the address of an element of **`table1`** whose index is in SI register

» We cannot use the OFFSET directive in this case

Where Are the Operands? (cont'd)

- **Displacement addressing mode**: instruction contains address of memory location (or register) + constant to add to the contents to that location to get final address of location containing data.



Where Are the Operands? (cont'd)

- **Indexed addressing mode**: used in arrays which can be accessed using index of memory location.

- Example: ADD R3, (R1+R2)
- Meaning: Reg[R3] ← Reg[R3]+ Mem[Reg[R1] + Reg[R2]] i.e., Take values in registers R1 and R2, add them to get address of memory location. Jump from that location by the amount equal to value in R3.

- See commonly used Addressing modes in book: **Computer Architecture** A Quantitative Approach *Fifth Edition* by morgan kaufmann page A-9.

COMPUTER TAXONOMY / CLASSIFICATION BASED ON INSTRUCTIONS

Taxonomy

- Computers are classified based on ISA architecture as:
  - Accumulator based machine
  - Stacked based machine
  - General purpose registers machine

Taxonomy: 1st class

- Accumulator based machine
  - It is based on accumulator register.
  - This register accumulates or holds the data like operand. In this case, second term of the operation is fetched from memory and result of some operation is stored in accumulator.
  - It is single address machine thus requires less memory access.
  - It was developed when memory was expensive

2nd class

- Stacked based machine
  - Works on the principle of LIFO (Last In First Out) principle.
  - POP operation takes out the data from the stack and after performing the operation, the data is inserted onto the top of the stack using PUSH operation.
  - For example let's perform addition operation on two numbers using stacked based machine:
  - (Show addition of two numbers: place two numbers in stack then access them using pop them one by one, perform add operation in ALU and push result back)

3rd class

- General purpose registers (GPR) machine
- A few number of registers are designed which could be addresses with a few number of bits. For example, 32 registers can be accessed using 5 bits only.

Machine Types

- **Register-register machine**: machines where both operands come from registers during ALU operation.
  - Note that in this machine category, we assume both operands are already fetched from memory to CPU registers.
  - In these machines ALU has no direct access to memory, it only gets data from CPU register.
  - These machines are called LOAD / STORE machines as here data (operands and result) are fetch and store from registers.
- **Register-memory machine**: one operand come from memory, other from register, it is called 1.5-address construct.
- **Memory-memory machine**: both operands come from memory. 2,3,4-address instructions.